

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	CH.SAISRI	Reg. No.:	192472367
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

1. Reference Resolution Using Multiple Constraints

Aim

To design an algorithm for reference resolution that identifies pronouns and selects the most appropriate antecedent using gender, number, recency, grammatical role, and semantic compatibility.

Given Discourse

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

Algorithm / Pseudocode

ALGORITHM Reference_Resolution

INPUT: Discourse containing entities and pronouns

OUTPUT: Resolved discourse and coreference chains

1. START
2. Read the input discourse.
3. Identify all named entities:
 - Meena
 - Kavya
 - Laptop
 - Project
4. Identify all referring expressions:
 - she
 - one
 - her
 - She
 - her
 - it
5. For each referring expression:
 - a. Generate possible antecedents.
 - b. Check gender agreement.
 - c. Check number agreement.
 - d. Check grammatical role.
 - e. Check recency.
 - f. Check semantic compatibility.
 - g. Check discourse coherence.
6. Assign a score to every possible antecedent.
7. Select the antecedent with the highest score.
8. Resolve the references:
 - First "she" → Kavya
 - "one" → Laptop
 - "her" → Kavya
 - Second "She" → Kavya
 - Second "her" → Meena
 - "it" → Laptop
9. Construct the coreference chains:
 - Kavya → she → She
 - Meena → her
 - Laptop → one → it
10. Replace the referring expressions with their selected antecedents.

11. Display the resolved discourse.

12. STOP

Reference Resolution Table

Referring Expression	Possible Antecedents	Valid Antecedent	Reason
she	Meena, Kavya	Kavya	Female and semantic context
one	laptop, project	Laptop	“one” refers to an object
her	Meena, Kavya	Kavya	Project belongs to Kavya
She	Meena, Kavya	Kavya	Recency and discourse continuity
her	Meena, Kavya	Meena	Kavya thanked Meena
it	laptop, project	Laptop	Laptop can be used

Resolved Discourse

“Meena gave Kavya a laptop because Kavya needed one laptop for her project. Kavya thanked Meena and started using the laptop.”

How the Algorithm Avoids Incorrect Resolution

The algorithm does not depend only on the nearest noun. It combines **gender, number, grammatical role, recency, semantic compatibility, and discourse coherence**. Therefore, when two antecedents have similar grammatical properties, semantic and discourse constraints help select the correct antecedent.

Result

The reference resolution algorithm correctly identifies the antecedents of the pronouns and produces a coherent resolved discourse.

2. Algorithm for Entity Chains and Discourse Coherence

Aim

To design an algorithm that identifies entity chains in a discourse and uses entity continuity to determine discourse coherence.

Given Text

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

Algorithm / Pseudocode

ALGORITHM Entity_Chain_Analysis

INPUT: Multi-sentence discourse

OUTPUT: Entity chains and coherence result

1. START

2. Read the discourse.

3. Divide the discourse into individual sentences.

4. Identify named entities and noun phrases.
5. Identify referring expressions such as:
 She
 it
 the computer
6. Compare each referring expression with previously mentioned entities.
7. If two expressions refer to the same entity, place them in the same entity chain.
8. Construct all entity chains.
9. Calculate entity continuity by checking how frequently entities are maintained across consecutive sentences.
10. If important entities continue across sentences, classify the discourse as coherent.
11. If entity chains are frequently broken, classify the discourse as less coherent.
12. Display entity chains.
13. Display coherence result.

14. STOP

Entity Identification

Sentence 1:

Anjali
Laptop

Sentence 2:

Laptop
Processor

Sentence 3:

She
Python
It

Sentence 4:

Anjali
Computer
Machine Learning Application
Entity Chains

Chain 1:

Anjali → She → Anjali

Chain 2:
Laptop → The laptop → it → the computer

Chain 3:
Processor

Chain 4:
Python

Chain 5:
Machine learning application

Entity Continuity

Entity	Sentence Occurrences	Continuity
Anjali	1, 3, 4	High
Laptop	1, 2, 3, 4	Very High
Processor	2	Low
Python	3	Low
Application	4	Low

Coherence Result

The discourse is **coherent** because the main entities, especially **Anjali and laptop**, are maintained across multiple sentences.

Effect of Breaking an Entity Chain

If an entity chain is broken, the reader may not know which object or person is being discussed.

For example:

Anjali bought a new laptop.
The car had a powerful processor.

The connection between the two sentences becomes unclear. Therefore, breaking entity chains reduces discourse coherence.

Result

Entity-chain analysis successfully identifies repeated references and shows that the given discourse is coherent.

3. Algorithm for Identifying Discourse Relations

Aim

To design an algorithm that identifies discourse relations between consecutive sentences using discourse markers and contextual meaning.

Given Discourse

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

Algorithm / Pseudocode

ALGORITHM Discourse_Relation_Identification

INPUT: Multi-sentence discourse

OUTPUT: Discourse relations and discourse structure

1. START
 2. Read the input discourse.
 3. Divide the discourse into individual discourse units.
 4. Identify explicit discourse markers.
 5. Search for markers such as:
 - because
 - therefore
 - as a result
 - after that
 - then
 6. Analyze the meaning of consecutive discourse units.
 7. If one event causes another event, assign CAUSE-EFFECT relation.
 8. If one event solves a previous problem, assign PROBLEM-SOLUTION relation.
 9. If events occur one after another, assign SEQUENCE relation.
 10. If one sentence provides additional information about another, assign ELABORATION relation.
 11. Construct the discourse structure.
 12. Display the identified relations.
 13. STOP
- Discourse Units
- D1: The server crashed unexpectedly.
- D2: As a result, users could not access the application.
- D3: The technical team restarted the server.
- D4: After that, the system became available again.

Discourse Markers

Marker	Relation
As a result	Cause–Effect
After that	Sequence

Identified Relations

$D1 \rightarrow D2$

CAUSE-EFFECT

$D2 \rightarrow D3$

PROBLEM-SOLUTION

$D3 \rightarrow D4$

SEQUENCE

Discourse Structure

Server crashed

|

| Cause

v

Users could not access application

|

| Problem

v

Technical team restarted server

|

| Sequence

v

System became available

Explanation

The first sentence introduces a **problem**. The second sentence describes its **effect**. The third sentence provides a **solution** by restarting the server. The fourth sentence describes the result after the solution. Therefore, the relations create a logical and coherent discourse.

Result

The algorithm correctly identifies **Cause–Effect**, **Problem–Solution**, and **Sequence** relations and establishes logical coherence between the sentences.

4. Algorithm for Dialogue Act Classification

Aim

To design an algorithm that identifies the communicative intention of each utterance and assigns an appropriate dialogue act.

Conversation

User: “Hello, I need help with my assignment.”

Agent: “Sure, what problem are you facing?”

User: “I do not understand machine learning.”

Agent: “I can explain the basic concepts to you.”

Algorithm / Pseudocode

ALGORITHM Dialogue_Act_Classification

INPUT: Conversation

OUTPUT: Dialogue-act sequence

1. START
2. Read each utterance in the conversation.
3. Preprocess the utterance.
4. Tokenize the utterance.
5. Identify important words and sentence patterns.
6. Check the communicative intention.
7. If the utterance contains a greeting,
assign GREETING.
8. If the user asks for help or information,
assign REQUEST.
9. If the utterance asks a question,
assign QUESTION.
10. If the speaker provides information,
assign INFORM.
11. If the speaker offers help,
assign OFFER.
12. If the speaker agrees or verifies something,
assign CONFIRMATION.
13. Store the dialogue acts in their
conversation order.
14. Display the dialogue-act sequence.
15. STOP

Dialogue Act Classification

Speaker	Utterance	Dialogue Act
User	Hello	Greeting
User	I need help with my assignment	Request
Agent	What problem are you facing?	Question
User	I do not understand machine learning	Inform
Agent	I can explain the basic concepts	Offer

Dialogue-Act Sequence

USER → GREETING + REQUEST

AGENT → QUESTION

USER → INFORM

AGENT → OFFER

How Dialogue Acts Help a Conversational Agent

Dialogue-act recognition helps the system understand the **intention** behind an utterance.

For example:

Greeting → Respond with greeting

Request → Provide help

Question → Give an answer

Inform → Understand user's situation

Offer → Allow user to accept or continue

Therefore, the agent can select an appropriate next response instead of responding only based on individual words.

Result

The algorithm correctly classifies the conversation into **Greeting, Request, Question, Inform, and Offer**, enabling the conversational agent to generate suitable responses.

5. Algorithm for Dialogue State Tracking

Aim

To design an algorithm for maintaining dialogue state while collecting information from a user during a flight-booking conversation.

Required Slots

Departure City

Destination City

Travel Date

Number of Passengers

Conversation

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Algorithm / Pseudocode

ALGORITHM Dialogue_State_Tracking

INPUT: Multi-turn conversation

OUTPUT: Updated dialogue state and next question

1. START

2. Initialize all dialogue slots:

Departure City = EMPTY
Destination City = EMPTY
Travel Date = EMPTY
Number of Passengers = EMPTY

3. Read the user's utterance.

4. Preprocess and tokenize the utterance.

5. Identify travel-related information.

6. If a destination is mentioned,
update Destination City.

7. If a departure city is mentioned,
update Departure City.

8. If a travel date is mentioned,
update Travel Date.

9. If number of passengers is mentioned,
update Number of Passengers.

10. Check all slots.

11. Identify the first missing required slot.

12. Generate a question for that missing slot.

13. Repeat the process when the user provides
additional information.

14. Stop when all required slots are filled.

15. Display the final dialogue state.

16. STOP

Step 1 – Initial State

Departure City = EMPTY
Destination City = EMPTY
Travel Date = EMPTY
Number of Passengers = EMPTY

Step 2 – User

“I want to book a flight.”

No slot information is provided.

Departure City = EMPTY
Destination City = EMPTY
Travel Date = EMPTY
Number of Passengers = EMPTY

Agent asks:

“Where would you like to travel?”

Step 3 – User

“I want to go to Delhi.”

Update:

Destination City = Delhi

Current state:

Departure City = EMPTY
Destination City = Delhi
Travel Date = EMPTY
Number of Passengers = EMPTY

Agent asks:

“From which city?”

Step 4 – User

“From Chennai.”

Update:

Departure City = Chennai

Final Dialogue State

DIALOGUE STATE

Departure City : Chennai
Destination City : Delhi
Travel Date : EMPTY
Number of Passengers : EMPTY

Next Question

Since the travel date is missing, the system should ask:

“What is your travel date?”

After receiving the date, the system should ask:

“How many passengers are travelling?”

Importance of Dialogue State Tracking

Dialogue state tracking maintains information from previous turns. The user does not need to repeat information already provided.

For example:

User: I want to go to Delhi.

System remembers:

Destination = Delhi

User: From Chennai.

System updates:

Departure = Chennai

This improves **coherence, accuracy, efficiency, and naturalness** in multi-turn conversations.

Result

The dialogue state tracking algorithm successfully maintains the booking information and identifies the missing slots required to complete the flight reservation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structu res	Pseudoc ode Structur e	Complete ness	Sub. Total	Total %
1	3	4	4	4	3	18	90
2	4	4	3	3	4	18	
3	3	4	4	3	4	18	
4	4	4	3	4	3	18	
5	3	3	4	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____